What is an AngularJS?

- AngularJS is an **open source Model-View-Controller framework which** is similar to the **JavaScript framework** that is used to build web applications.
- It is a library **written in JavaScript.**
- This framework has been developed by a group of developers from **Google** itself.
- Because of the sheer support of Google and ideas from a wide community forum, the framework is always kept up to date. Also, it always incorporates the latest development trends in the market.
- **AngularJS changes static HTML to dynamic HTML.** It extends the ability of HTML by adding built-in attributes and components and also provides an ability to create custom attributes using simple JavaScript.
- It can be freely used, changed and shared by anyone.
- Angularjs is **developed by Google**.
- It is an excellent framework for building **single phase applications.**
- AngularJS is a **structural framework for dynamic web applications.**
- It can be added to an HTML page with a <script> tag.
- AngularJS is entirely based on HTML and JavaScript, so there is **no need to learn another syntax or language.** AngularJS **extends HTML attributes with Directives**, and **binds data to HTML with Expressions**.
- AngularJS is distributed as a JavaScript file, and can be added to a web page with a script tag:

```
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.4/angular.min.js"></script>
```

Some of the live examples of popular apps that use AngularJS

- Video Streaming Apps
- User-Review Applications
- Travel Apps: JetBlue's website
- Weather Apps
- User Generated Content Portals: Freelancer.com
- eCommerce
- Mobile Commerce: MallZee
- Social Apps : LinkedIn

Advantages of AngularJS:

1. **An intuitive framework makes it easy to organized your code:**
    - AngularJS is more intuitive as it makes **use of HTML as a declarative language**. Moreover, it is less hard for reorganizing.
2. **Built by google:**
    - AngularJS has been developed as well as maintained by dedicated Google engineers. This means that there is a huge community out there for you to

learn from. Apart from that, there are engineers that can help you tackle any challenges you face on the way. It also means that clients get what they want.

3. **AngularJS code is unit testable:**
   - AngularJS is unit testing ready, and that is one of its most compelling advantages. The designers at Google not only developed Angular but also developed a testing framework called "Karma" which helps in designing unit tests for AngularJS applications.
4. **saves you hundreds of lines of code:**
   - It is possible to do two-way data binding with AngularJS, meaning you can make any data related changes and it would immediately be propagated to the corresponding views and when any change is made in the view, that would happen in the underlying model as well.
5. **Comprehensive:**
   - AngularJS is a **comprehensive solution for rapid front-end development**. It **does not need any other plugins or frameworks**. Moreover, there are a range of other features that include Restful actions, data building, dependency injection, enterprise-level testing, etc.

6. **Responsive**: It provides data binding capability to HTML. Thus, it gives user a rich and responsive experience.
7. **Dependency Injection:**
   AngularJS **uses dependency injection** and make use of separation of concerns:
   - Angular **has its own dependency injection framework**, and you really can't build an Angular application without it.
   - Dependency Injection (DI) is a way to **create objects that depend upon other objects**. A Dependency Injection system supplies the dependent objects (called the dependencies) when it creates an instance of an object.
   - Dependency Injection in AngularJS can be defines as the **software design pattern which defines the way the software components are dependent on each other**. AngularJS provides a set of components that can be injected in the form of dependencies such as factory, value, constant, service, and provider.
   - dependencies, or make them yourself, but would be made and brought before you.
   - In order to ensure you have AngularJS's core features, you can simply add that parameter and customize it as well.
8. AngularJS provides **reusable components**.
9. With AngularJS, the developers can achieve **more functionality with short code.**
10. **Routing** – Angular can take care of routing which means moving from one view to another. This is the key fundamental of single page applications; wherein you can move to different functionalities in your web application based on user interaction but still stay on the same page.
11. **Great MVC:**
    - frameworks are tools that would help you manage web page interactions and data collections. Angular JS provides the best "lightweight solution" for creating a balance between the client logic and server.

- In AngularJS, views are pure html pages, and controllers written in JavaScript do the business processing.

AngularJS Application:

Include angular.js file in the head section (you have learned how to download angular library in the previous section). You can take a reference from the CDN also. (all the examples in this tutorials will use CDN reference.)

```
<script src ="http://ajax.googleapis.com/ajax/libs/angularjs/1.0.4/angular.js">        </script>
```

1. **Template:**
   In AngularJS, a template is HTML with additional markups. AngularJS compiles templates and renders the resultant HTML.

2. **Directives** are extensions of HTML markups. They could take the form of attributes, element names, CSS class and or even HTML comments.
   - **ng-app:** is a directive that bootstraps AngularJS. It designates the caller element as the root. It's usually placed on <html> or <body>. This directive defines and links an AngularJS application to HTML.
     - ng-app directive auto initializes an AngularJS framework.
     - it is required to tell which part of HTML contains the AngularJS app. You can do this by adding the ng-app attribute to the root HTML element of the AngularJS app. You can either add it to the html element or the body element as shown below.

```
<body ng-app = "">
  ...

</body>
```

     - we can also initialize AngularJS manually without using ng-app directive.

```
<script>
                    angular.element(document).ready(function (){
                    angular.bootstrap(document);
            });
</script>
```

   - **ng-model:** is a directive that binds usually form elements. For instance, input, select, checkboxes, textarea. They keep data (model) and visual elements (HTML) in sync.
     ➔ This directive binds the values of AngularJS application data to HTML input controls.
   - **Ng-bind/ {{ name }} :** {{ }} are a way of binding models to elements in HTML. In the example above, the ng-model name is bound to the placeholder {{ name }}.

3. AngularJS Expression:
   - Expressions are used to bind application data to HTML. Expressions are written inside double curly braces such as in {{expression}}. Expressions

behave similar to ng-bind directives. AngularJS expressions are pure JavaScript expressions and output the data where they are used.

- Syntax:
    - A simple example of an expression is {{5 + 6}}.
- Angular.JS expressions are used to bind data to HTML the same way as the ng-bind directive. AngularJS displays the data exactly at the place where the expression is placed.
- AngularJS expression is like JavaScript code expression except for the following differences:
    - AngularJS expression cannot contain conditions, loops, exceptions or regular expressions e.g. if-else, ternary, for loop, while loop etc.
    - AngularJS expression cannot declare functions.
    - AngularJS expression cannot contain comma or void.
    - AngularJS expression cannot contain return keyword.
- Expressions are written inside double braces {{expression}}. They can also be written inside a directive:

  ng-bind="5*7".

- AngularJS expressions are very similar to JavaScript expressions. They can contain literals, operators, and variables. For example:

  {{ 4+5}} or {{ firstName + " " + lastName }}

AngularJS Expressions	JavaScript Expressions
1. AngularJS expressions can be written inside HTML.	1. **JavaScript** expressions cannot be written inside HTML.
2. Do not support conditionals, loops, and exceptions	2. Support conditionals, loops, and exceptions
3. Support filters	3. Does not support filters
4. Contain literals, operators, and variables.	4. Contain literals, operators, and variables.

How AngularJS Integrates with HTML

✓ In order to get your first Angular.js app up and running you need to include the Angular Javascript file via script tag and make use of the ng-app directive.

```
<html>
  <head>
    <script src="angular.js">
    </script>
  </head>
  <body ng-app="">
    <p>This is your first angular expression: {{1 + 2}} </p>
  </body>
</html>
```

✓ The ng-app directive indicates the start of AngularJS application.

Binding a Text Input to an Expression

- ✓ You want user input to be used in another part of your HTML page
- ✓ Use AngularJs ng-model directive to bind the text input to the expression.

```
Enter your name: <input type="text" ng-model="name"></input>
<p>Hello {{name}}!</p>
```

- ✓ Assigning "name" to the ng-model attribute and using the name variable in an expression will keep both in sync automatically. Typing in the text input will automatically reflect these changes in the paragraph element below.
- ✓ The ng-model directive creates a model variable named name, which can be used with the HTML page and within the div having ng-app directive.
- ✓ The ng-bind then uses the name model to be displayed in the HTML tag whenever user enters input in the text box.

AngularJS Directives

- AngularJS lets you extend HTML with new attributes called **Directives**.
- AngularJS has a set of built-in directives which offers functionality to your applications.
- AngularJS also lets you define your own directives.
- AngularJS directives are extended HTML attributes with the prefix ng-.
    - ng-app directive starts an AngularJS application.
    - ng-init directive initializes application data.
    - ng-model directive binds the value of HTML controls (input, select, textarea) to application data.
    - ng-repeat directive repeats HTML elements for each item in a collection.

ng-app directive

- ng-app directive starts an AngularJS Application.
- It defines the root element.
- It automatically initializes or bootstraps the application when web page containing AngularJS Application is loaded.
- It is also used to load various AngularJS modules in AngularJS Application.
- In following example, we've defined a default AngularJS application using ng-app attribute of a div element.

```
<div ng-app = "">
  ...

</div>
```

ng-init directive

- ng-init directive initializes an AngularJS Application data.
- It is used to put values to the variables to be used in the application.
- In following example, we'll initialize an array of countries. We're using JSON syntax to define array of countries.

```
<div ng-app = "" ng-init = "msg='CMPICA' ">
  ...
</div>
```

ng-model directive

- This directive binds the values of AngularJS application data to HTML input controls. In following example, we've defined a model named "name".

```
<div ng-app = "">
  ...
  <p>Enter your Name: <input type = "text" ng-model = "msg"></p>
  <p> Welcome: {{msg}}</p>
</div>
```

Example

For single value

```
<div ng-app="" ng-init=" msg='Hello World'">
      Message: <input type="text" ng-model="msg"> <br>
      You wrote: {{ msg }}
</div>
```

For multiple value

```
<div ng-app="" ng-init="quantity=10; price=5">
      Quantity: <input type="number" ng-model="quantity">
      Costs:   <input type="number" ng-model="price">
      Total in dollar: {{quantity * price}}
</div>
<div ng-app="" ng-init="quantity=10; price=5; msg='Hello World'">
      Quantity: <input type="number" ng-model="quantity">
      Costs:   <input type="number" ng-model="price">
      Total in dollar: {{ quantity * price }} <br>
      Message: <input type="text" ng-model="msg">
      You wrote: {{ msg }}
</div>
```

ng-repeat directive

- ng-repeat directive repeats html elements for each item in a collection. In following example, we've iterated over array of objects.

- The ng-repeat directive has a set of special variables which you are useful while iterating the collection. These variables are as follows:
  - $index
  - $first
  - $middle
  - $last

```
<div ng-app = "" ng-init="person=['harsh', 'Saumil', 'Khevna']">
  ...
```

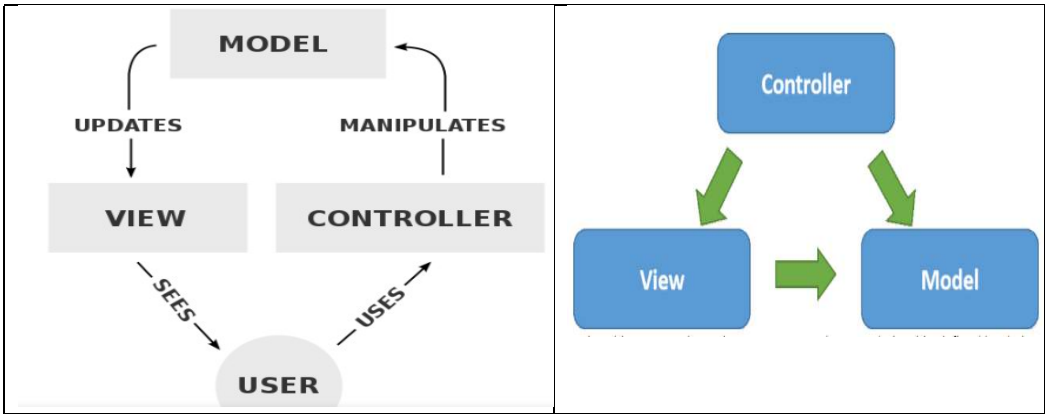
```
<p>List of Persons :</p>

  <ol>
    <li ng-repeat = "i in person">
      {{i}}
    </li>
  </ol>

<p>List of Subject :</p>

  <ol>
    <li ng-repeat = "i in [‘sub1’, 'sub2','sub3']">
      {{i}}
    </li>
  </ol>

</div>
```

Array Object

```
<div ng-app="" ng-init="names=[
                    {name:'Jani',country:'Norway'},
                    {name:'Hege',country:'Sweden'},
                    {name:'Kai',country:'Denmark'}]">

            <h1>{{ names[1].name }} </h1>

        <p>Looping with objects:</p>
        <ul>
          <li ng-repeat="x in names">
          {{ x.name + ', ' + x.country }}</li>
        </ul>

      <ol>
        <li ng-repeat="x in [
          {Name:'Banana',price:20},
          {Name:'Apple',price:50},
          {Name:'Orange',price:30}]">
         {{'Fruit Name= '+ x.Name + ' and ' + 'Price= '+x.price}}</li> </ol>
</div>
```

AngularJS MVC:

- MVC is popular because it isolates the application logic from the user interface layer and supports separation of concerns.
- The **controller receives all requests for the application and then works with the model to prepare any data needed by the view**.
- The **view then uses the data prepared by the controller to generate a final presentable response**.
- In AngularJS the MVC pattern is implemented in JavaScript and HTML.
  - ✓ The view is defined in HTML
  - ✓ while the model and controller are implemented in JavaScript.
- The MVC abstraction can be graphically represented as follows.



- The model is only aware of itself.
- The view is aware of the model, as it's responsible for rendering data contained in the model and invoking actions (methods) on the model.
- The job of the controller is to create and populate the model and hand it over to the view. The controller needs to know about the model, and how to resolve the view and provide it the model.

A Model View Controller pattern is made up of the following three parts:

- **Model:**
  - The model is **responsible for managing application data**
  - **lowest level of the pattern responsible for maintaining data.**
  - It responds to the request from view and to the instructions from controller to update itself

```
<script>
   $scope.message = "Good Morning"
</script>
```

- **View:**
  - It is responsible for **displaying all or a portion of the data** to the user.
  - View is display of model that is your data. It uses double curly brackets and it is lot like using JavaScript template library

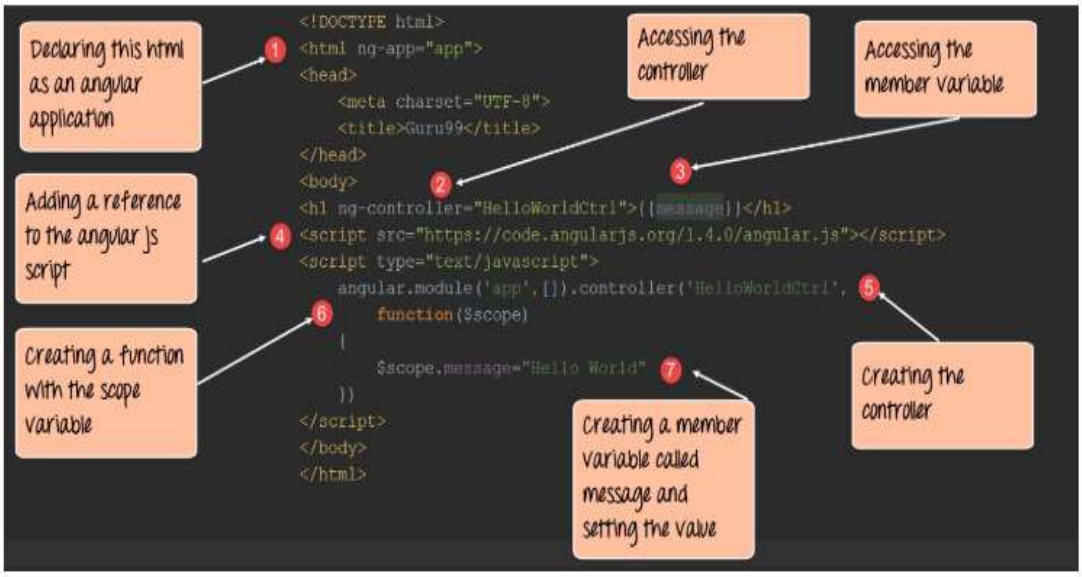
    <h1> {{message}} </h1>
- **Controller:**
  - It is a **software Code that controls the interactions between the Model and View.**
  - The controller receives input, validates it, and then performs business operations that modify the state of the data model.
  - Controller gives you control over model and view which means it controls how data is retrieved and displayed to end-user.
  - It wraps MODEL in this controller function: (You should add MODEL inside this function which is shown above)

```
function GreetingController($scope)
{
        $scope.message = "Good Morning"; //model
}
```

What is $scope?

- ✓ is glue between the view and the controller within an angularJS application
- ✓ The scope is a JavaScript object which basically binds the "controller" and the "view". One can define member variables in the scope within the controller which can then be accessed by the view.
- ✓ scope is a JavaScript object with properties and methods, which are available for both the view and the controller.

There are several ways that these components can be put together in AngularJS but the simplest form starts with the view.



Code Explaination:

1. The "ng-app" keyword is used to denote that this application should be considered as an angularjs application. Any name can be given to this application.
2. The controller is what is used to hold the business logic. In the h1 tag, we want to access the controller, which will have the logic to display "HelloWorld", so we can say, in this tag we want to access the controller named "HelloWorldCtrl".
3. We are using a member variable called "message" which is nothing but a placeholder to display the "Hello World" message.
4. The "script tag" is used to reference the angular.js script which has all the necessary functionality for angular js. Without this reference, if we try to use any AngularJS functions, they will not work.
5. "Controller" is the place where we are actually creating our business logic, which is our controller. The specifics of each keyword will be explained in the subsequent chapters. What is important to note that we are defining a controller method called 'HelloWorldCtrl' which is being referenced in Step2.
6. We are creating a "function" which will be called when our code calls this controller. The $scope object is a special object in AngularJS which is a global object used within Angular.js. The $scope object is used to manage the data between the controller and the view.
7. We are creating a member variable called "message", assigning it the value of "HelloWorld" and attaching the member variable to the scope object.

$scope in angularJS

- The **$scope** in an AngularJS is a built-in object, which contains application data and methods. You can create properties to a $scope object inside a controller function and assign a value or function to it.
- The $scope is glue between a controller and view (HTML). It transfers data from the controller to view and vice-versa.

- As we have seen in the controller section, we can attach properties and methods to the $scope object inside controller function. The view can display $scope data using an expression, ng-model, or ng-bind directive, as shown below.
- AngularJS creates and injects a different $scope object to each controller in an application. So, the data and methods attached to $scope inside one controller cannot be accessed in another controller.
- With the nested controller, child controller will inherit the parent controller's scope object. Therefore, child controller can access properties added in parent controller but parent controller cannot access properties added in child controller.

$rootScope

- The $rootScope is the top-most scope. An app can have only one $rootScope which will be shared among all the components of an app. Hence it acts like a global variable. All other $scopes are children of the $rootScope.
- An AngularJS application has a single $rootScope. All the other $scope objects are child objects.
- The properties and methods attached to $rootScope will be available to all the controllers.
- The following example demonstrates the $rootScope and $scope object.

AngularJS Data Binding

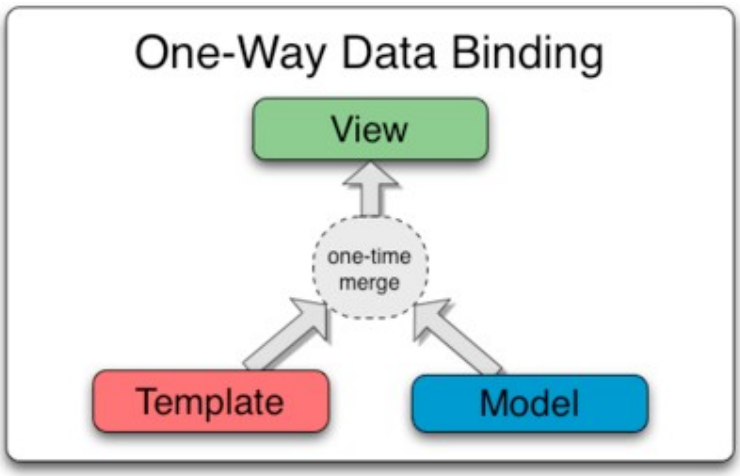
- The **data binding** is the data synchronization processes that work between the model and view components.
- In Angular, **model treat as source of application and view is the projection of angular model.**
- In angularJS when model data got changed that time the view data will change automatically and vice versa.

- We have two types of data bindings available in angularjs those are

  1. One-Way data binding
  2. Two-Way data binding

AngularJS One-Way Data Binding:

- In **One-Way** data binding, view (UI part) not updates automatically when data model changed and we need to write custom code to make it updated every time.
- It's not a synchronization processes and it will process data in one way that will be like as shown following image.



AngularJS One-way Data Binding Example

Following is the example of using one-way data binding in angularJS application.

```
<script type="text/javascript">
        var app = angular.module('angulartwobindapp', []);
        app.controller('angulartwobindingCtrl', function ($scope) {
        $scope.name = 'Welcome to CMPICA';
        });
</script>
</head>
<body ng-app="angulartwobindapp">
        <div ng-controller="angulartwobindingCtrl">
        <p> Message:   {{ name }}
        </p>
        </div>
</body>
```

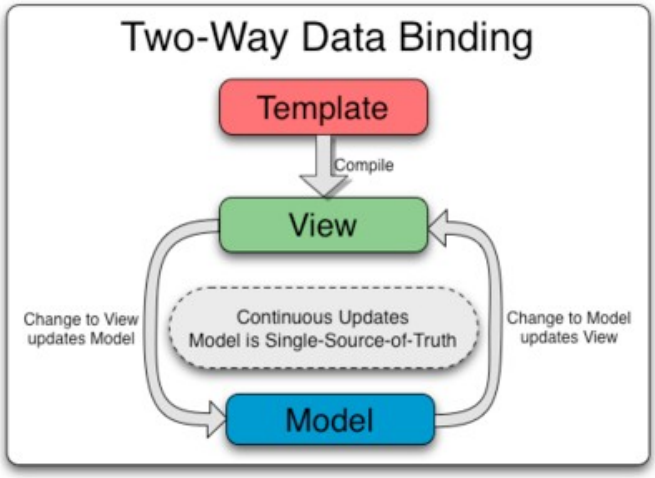
- Here if you observe above code we are binding model values to html elements using data bindings but html elements it won't change the values in model its one way data binding.

Output of AngularJS One-way Data Binding

```
Message: Welcome to CMPICA
```

AngularJS Two-way Data Binding:

- In **Two-way** data binding, view (UI part) updates automatically when data model changed.
- Its synchronization processes and two-way data binding that will be like as shown following image.



- We can achieve this two-way data binding using ng-model directive.
- If we use ng-model directive in html control it will update value automatically whenever data got changed in input control.

Angular Module:

- A module defines the **application functionality that is applied to the entire HTML** page using the ng-app directive.
- A module in AngularJS is a container of the different parts of an application such as controller, service, filters, directives, factories etc.
- Modules are used to separate logics say services, controllers, application etc. and keep the code clean.
- We define modules in separate js files and name them as per the module.js file.
- It defines functionality, such as services, directives, and filters, in a way that makes it easy to reuse it in different applications.
- **Types:**
  - Application module: used to initialize an application with controller(s). (ng-app)
  - Controller Module: used to define the controller. (ng-controller)

Creating an Application Module:

- The angular object's module() method is used to create a module. It is also called AngularJS function angular.module
- Application module is created by using the AngularJS function angular.module

```
<div ng-app="myApp">...</div>
<script>
    var moduleDisplay = angular.module("myApp", []);
</script>
```

- The "myApp" parameter refers to an HTML element in which the application will run.
- The first argument is the module name myApp.
- The second argument is an array of module names that this module depends on. The angular.module() method returns specified module object if no dependency is specified. Therefore, specify an empty array even if the current module is not dependent on other module.
- The second parameter is used to define the module's dependencies - i.e., a list of modules (module names, to be precise) that should be already loaded by the injector before the current module is loaded.
- Empty square brackets denote that this module does not have any dependencies. This array generally contains dependent modules.

Creating a Controller Module:

- Add a controller to your application, and refer to the controller with the ng-controller directive:

```
<body>

    <div ng-app="myApp" ng-controller="myCtrl">
    {{ firstName + " " + lastName }}
    </div>
    <script>
    var moduleDisplay = angular.module("myApp", []);   //mainAppModule.js

    moduleDisplay.controller("myCtrl", function($scope) { //mainAppController.js
       $scope.firstName = "John";
       $scope.lastName = "Doe";
    });
    </script>
    </body>
```

Modules and Controllers in Files

- It is common in AngularJS applications to put the module and the controllers in JavaScript files.
- In this example, "myApp.js" contains an application module definition, while "myCtrl.js" contains the controller:

```
<div ng-app="myApp" ng-controller="myCtrl">
{{ firstName + " " + lastName }}
</div>

<script src="myApp.js"></script>
<script src="myCtrl.js"></script>
```

- **myApp.js**

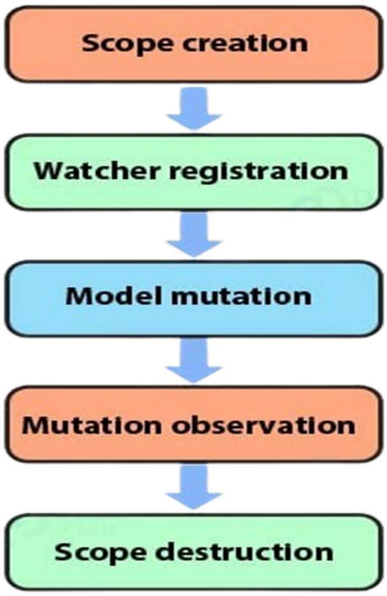
```
var app = angular.module("myApp", []);
```

- **myCtrl.js**

```
app.controller("myCtrl", function($scope) {
   $scope.firstName = "John";
   $scope.lastName= "Doe";
});
```

Life Cycle of Angularjs scope

In the flow of **JavaScript code**, whenever an event is received by browser then it executes a JavaScript callback corresponding to it. And after the completion of callback, DOM objects are re-rendered by a browser. If any JavaScript code is executed outside the context of angularJS by browser, then angularJS remain unaware with the changes in the model. To detect the model change in angularJS $apply API is used.



1. **Creation –** During bootstrap of an application using **$injector**, root scope is created. And during the linking of a template, many of the directives creates new child scope.

```
var app = angular.module('myApp',[]);
app.run(function($rootScope){
        $rootScope.globalData='MCA'
    })
```

2. **Watcher Registration –** Model values can propagate to DOM using a $watch. These watches propagate model changes automatically to the DOM elements. You can also register your own watch functions on a scope value by using the $watch() function.

```
app.controller('myCtrl',function($scope,$rootScope){
        $scope.innerData = $rootScope.data;
       $scope.$watch('myVal', function (newvalue, oldvalue) {
           $scope.new = newvalue;
           $scope.old = oldvalue;
        })

    })
```

3. **Model Mutation –** To observe mutation in a proper way, an API known as $apply is used. When the changes in the scope are done inside the code, for e.g. in services and controllers, angular internally calls the function $apply(), but when the changes in the scope are done outside the code, then the $apply() function is externally called to update DOM and model properly. $apply is used to estimate the parameters and expressions which lie outside the scope.

```
// change any model outside of the Angular context,
        inform AngularJS of the changes by calling $apply() manually
document.getElementById('changeBtn').addEventListener('click',()=>{
    $scope.$apply(()=>{
    $scope.innerData = 'This is change'
});
```

4. **Mutation Observation –** After $apply, a $digest is performed on the root scope in angularJS. In this phase, the $digest function is executed to run all the watches for the change in the model. If there is a change in the code, then the $digest() function calls the $watch listener function to update DOM elements.
5. **Scope Destruction –** If child scope is no longer in use, then it should be destroyed. Child scope creator is responsible to destroy the child scope if no longer in use. Using API $scope.$destroy() child scope creator can do so. Destroying an unused child scope will release memory that is being used by it and stop model propagation too.

```
$scope.destroyData = function(){
        $scope.$destroy();
}
```

Types of Directives in angularJS

There are two main types of directives in AngularJS.

1. **Built-In directives** - These directives in AngularJS are already predefined. We can directly use them for different functionalities in our application.
2. **Custom directives** – Custom directives are made by the user. They are not predefined. We can define them based on their functionality.

Built-In Directives

- Built-in directives are used to manage the elements like forms, lists, and styles of the application. To put it more simply, we can say that built-in directives are used to manage the elements that the user can see on the screen.
- Based on the functions of directives, built-in directives can further be classified in three diverse types.
    1. **Components:** Component directives in angular with the template.
    2. **Structural directives:** These directives change the DOM layout by adding and removing DOM elements.
    3. **Attribute directives:** These directives change the appearance or behaviour of an element, component, or another directive.

  Here are some examples of built-in directives in angularjs
    1. ng-init
    2. ng-bind
    3. ng-model
    4. ng-repeat
    5. ng-if
    6. ng-disable
    7. ng-readonly
    8. ng-class
    9. ng-style
    10. ng-for
    11. ng-switch

Custom Directives

- Custom directives are **user-defined directives** in AngularJS. These allow the user to use desired functions to extend the function ality of HTML. Also, new directives can be created to **manipulate the HTML behaviour**. Thus, users can create directives in angular.
- Users can use the directive function to define those custom directives, and it can replace the element for which it can be used. AngularJS has a lot of popular built-in directives; still, sometimes custom directives are required.

Introduction to the ways to implement custom directives in AngularJS.

1. **Element directives:** It is activated when finding a matching HTML element in the HTML template. For example: <directive-name></directive-name>
2. **Attribute directives:** It is activated when finding a matching HTML attribute in the template. For example: <div directive-name>
3. **CSS Class directive:** It is activated when finding a matching CSS class in the template. For example:
4. **Comment directives:** It is activated when finding a matching HTML comment. For example: <!-- Directive: directive-name expression; -->

Commonly used properties

1. Restrict:

- It specifies how a directive is implemented in angular app. There are 4 restrict options.
- restrict: 'A' – attribute (default one) | <div my-directive>
- restrict: 'C' – Class |
- restrict: 'E' – element | <my-directive></my-directive>
- restrict: 'M' – Comment | <!– Directive : my-directive expression; –>
- By default restrict "EA"
- You can also specify multiple restrict like as follows:
- restrict: 'EC'

2. Scope

- Scope accesses data or methods inside template and link function. By default, the directives do not produce their own scope without explicitly set.
- Different types of scopes are able to define which are as follows:
  - scope: true – Get new scope
  - scope: false – Use its parent scope
  - scope: {} – Get new isolated scope that doesn't inherit from parent and exists on its own

3. Template

- It specifies the HTML content that will be generated when directive is compiled.

4. TemplateUrl

- It specifies the path of template that should be used by directives.

How to Add Directives

- New directives are created by using the .directive function. To invoke a new directive, we have to make an HTML element that has the same tag name as the new directive.
- While naming a directive, we must use a camel case name like myNewDirective, but while invoking it, we must use the name separated with – like my-new-directive.

```
body ng-app=”myApp”>
<my-new-directive></my-new-directive>
<script>
var app = angular.module(“myApp”,[]);
App.directive(“myNewDirective”, function(){
return{
template : “<h1>Made by a directive! </h1>”
};
});
</script> </body>
```

We can invoke a directive by using different methods. That are

1. Element name
2. Attribute
3. Class
4. Comment

There are some legal restrict values.

1. E – for Element name
2. A – for Attribute
3. C – for Class
4. M – for Comment

The default restrict value is EA. This means that both Element names and attribute names can invoke the directive.

Step 1: Building a reusable component

Directives in AngularJS can also be used to create a reusable component. Here is an example of a user box component.

UserBox.js

```
angular.module('simpleDirective', []).directive('userBox', function() {
        return {
                scope: {
                        username: '=username',
                        reputation: '=reputation'
                    },
                templateUrl: '/path/to/app/directives/user-box.html'
            };
});
```

Controller.js

```
var myApp = angular.module('myApp', ['simpleDirective']);
        myApp.controller('Controller', function($scope) {
        $scope.user = "John Doe";
        $scope.rep = 1250;
});
```

MyPage.js

```
<html lang="en" ng-app="myApp">
<head>
<script src="/path/to/app/angular.min.js"></script>
<script src="/path/to/app/controllers/Controller.js"></script>
<script src="/path/to/app/directives/userBox.js"></script>
</head>
<body>
<div ng-controller="Controller">
<user-box username="user" reputation="rep"></user-box>
</div>
</body>
</html>
```

UserBox.html

```
<div>{{username}}</div>

<div>{{reputation}} reputation</div>
```

AngularJS Conditions

ng-if Directive

ng-if**: -** directive is condition based directive which removes or recreates the DOM portion according to condition. It evaluates expression as true or false.

- If ng-if evaluates to a false it will remove from the DOM element and
- if it is true, then element is reinserted into the DOM.

Supported by all HTML elements.

Note: - The ng-if directive is different from the ng-hide, which hides the display of the element, where the ng-if directive completely removes the element from the DOM.

Syntax

<element ng-if="*expression*"*></element>*

Value	Description
expression	An expression that will completely remove the element if it returns false. If it returns true, a copy of the element will be inserted instead.

Example of ng-if (Even or Odd)

```
<body ng-app>
   Enter your number: - <input type="text" ng-model="n1" placeholder="9">
 <div ng-if="n1%2=='0'">
    <p>Even Number</p>
 </div>
 <div ng-if="n1%2!='0'">
    <p>Odd Number</p>
 </div>
</body>
```

Example (Uncheck a checkbox to remove a section)

```
<body ng-app>
Keep HTML: <input type="checkbox" ng-model="myVar" ng-init="myVar=true">
                <div ng-if="myVar">
                        <h1>Welcome</h1>
                        <p>Welcome to my home.</p>
                </div>
</body>
```

myVar having two values true & false. When checkbox checked at that time myvar==true and when its unchecked then myVar==false

Example (object variable in controller)

```
<body>
      <script>
            var app = angular.module('myApp', []);
            app.controller('myctrl', function($scope) {
              $scope.comp ={
                 code:'CS',
                 name: 'code-sample.com'
               };
            });
      </script>
     <div ng-app="myApp" ng-controller="myctrl">
         <h2>ng-if else in angularjs</h2>
         <hr/>
           <div>
         <div ng-if="comp.code=='CS'">
            If code is = <span style="color:green;">{{comp.code}}</span> ,
            than company is = <a href="#">{{comp.name}}</a>
         </div>
         <div ng-if="comp.code=='CS1'">
            code-sample.xyz
         </div>
      </div>
   </div>
</body>
```

ng-switch Directive

ng-switch: - directive will behave similar to the switch case in general programming scenario.

- ng-switch directive lets you hide/show HTML elements depending on an expression
  - Child elements with the ng-switch-when directive will be displayed if it gets a match, otherwise the element, and its children will be removed.
  - You can also define a default section, by using the ng-switch-default directive, to show a section if none of the other sections get a match.

Supported by all HTML elements.

Syntax

```
<element ng-switch="expression">
        <element ng-switch-when="value">
                design
        </element>
  <element ng-switch-when="value"></element>
  <element ng-switch-default></element>
</element>
```

Value	Description
Expression	An expression that will remove elements with no match, and display elements with a match.

```
<body ng-app="">
      Choose your favorite topic:
        <div ng-switch="myVar">
          <div ng-switch-when="dogs">
            <h1>Dogs</h1>
            <p>Welcome to a world of dogs.</p>
          </div>
          <div ng-switch-when="tuts">
            <h1>Tutorials</h1>
            <p>Learn from examples.</p>
          </div>
          <div ng-switch-default>
            <h1>Switch</h1>
            <p>Select topic from the dropdown, to switch the content of this
                  DIV.</p>
          </div>
        </div>
</body>
```

Example (Array variable in controller)

```
<body ng-app="MyApp">
   <script>
```

```
        var app = angular.module("MyApp", []);
        app.controller("myCtrl", function ($scope) {
        $scope.items = ['one', 'two', 'three', 'four'];
        $scope.selectval = "one";
        });
    </script>
      <div ng-controller="myCtrl">
         Select Item:
         <select ng-model="selectval" ng-options="item for item in items">
        </select>
        <div ng-switch ="selectval">
           <div ng-switch-when="one"><b>DIV1</b>: You Selected value
                                one</div>
           <div ng-switch-when="two"><b>DIV2</b>: You Selected value
                                two</div>
           <div ng-switch-default>You are in default switch case.</div>
        </div>
      </div>
</body>
```

AngularJS HTML DOM

In AngularJS, some directives can be used to bind application data to attributes of HTML DOM elements.

These directives are:

Directive	Description
ng-disabled	It disables a given control.
ng-show	It shows a given control.
ng-hide	It hides a given control.

Syntax

<element directives-name="*expression*"*></element>*

Value	Description
Directives-name	ng-disabled / ng-show / ng-hide
Expression	An expression that will show the element only if the expression returns true.

ng-disabled Directive

The ng-disabled directive binds AngularJS application data to the disabled attribute of HTML elements. In the below code, it binds a model to a checkbox.

```
<input type = "checkbox" ng-model = "enableDisableButton">Disable Button
    <button ng-disabled = "enableDisableButton">  Click Me!  </button>
```

ng-show Directive

The ng-show directive shows the specified HTML element if the expression evaluates to true, otherwise the HTML element is hidden.

```
Show HTML: <input type="checkbox" ng-model="myVar">
<div ng-show="myVar">
<h1>Welcome</h1>
<p>Welcome to my home.</p>
</div>
```

ng-hide Directive

he ng-hide directive hides the HTML element if the expression evaluates to true.

```
Hide HTML: <input type="checkbox" ng-model="myVar">
<div ng-hide="myVar">
<h1>Welcome</h1>
<p>Welcome to my home.</p>
</div>
```

example:

```
<h2>AngularJS Sample Application</h2>
<div ng-app = "">
    <input type = "checkbox" ng-model = "enableDisableButton"> Disable Button
    <button ng-disabled = "enableDisableButton">Disable Me!</button> <br>
    <input type = "checkbox" ng-model = "showHide1">Show Button
    <button ng-show = "showHide1">Show Me!</button> <br>
    <input type = "checkbox" ng-model = "showHide2">Hide Button
    <button ng-hide = "showHide2">Hide Me!</button> <br>
    <button ng-click = "clickCounter = clickCounter + 1"> Click Me!</button>
    <p>Total click: {{ clickCounter }}</p>
</div>
```

ng-options Directive

ng-options :- directive fills a <select> element with <options>.

- The ng-options directive uses an array to fill the dropdown list.
- In many cases it would be easier to use the ng-repeat directive, but you have more flexibility when using the ng-options directive.

```
<div ng-app="myApp" ng-controller="myCtrl">
<select ng-model="selectedName" ng-options="item for item in names"></select>
</div>
<script>
```

```
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
   $scope.names = ["Email", "Tobias", "Linus"];
});
</script>
```

ng-options vs ng-repeat

- ng-repeat directive repeats a block of HTML code for each item in an array, it can be used to create options in a dropdown list.
- but the ng-options directive was made especially for filling a dropdown list with options, and has at least one important advantage:
- Dropdowns made with ng-options allows the selected value to be an **object**, while dropdowns made from ng-repeat has to be a string.
- Assume you have an array of objects:

```
$scope.cars = [
   {model : "Ford Mustang", color : "red"},
   {model : "Fiat 500", color : "white"},
   {model : "Volvo XC90", color : "black"}
];
```

- The ng-repeat directive has its limitations; the selected value must be a string:

```
<select ng-model="selectedCar">
<option ng-repeat="x in cars" value="{{x.model}}">{{x.model}}</option>
</select>
<h1>You selected: {{selectedCar}}</h1>
```

- When using the ng-options directive, the selected value can be an object:

```
<select ng-model="selectedCar" ng-options="x.model for x in cars">
</select>
<h1>You selected: {{selectedCar.model}}</h1>
<p>Its color is: {{selectedCar.color}}</p>
```

- When the selected value can be an object, it can hold more information, and your application can be more flexible.

ng-change Directive

ng-change :- directive tells AngularJS what to do when the value of an HTML element changes.

- The ng-change directive requires a ng-model directive to be present.
- The ng-change directive from AngularJS will not override the element's original onchange event, both the ng-change expression and the original onchange event will be executed.
- The ng-change event is triggered at every change in the value. It will not wait until all changes are made, or when the input field loses focus.

- The ng-change event is only triggered if there is an actual change in the input value, and not if the change was made from a JavaScript.

```
<div ng-app="myApp" ng-controller="myCtrl">
        <select ng-change="myFunc()" ng-model="myValue">
                <option>One </option>
                <option>two </option>
                <option>Three </option>
        </select>
        <p>The input field has changed {{count}} times.</p>
</div>
<script>
angular.module('myApp', []).controller('myCtrl', ['$scope', function($scope)  {
        $scope.count = 0;
        $scope.myFunc = function() {
        $scope.count++;
    };
  }]);
</script>
```

AngularJS Events

You can add AngularJS event listeners to your HTML elements by using one or more of these directives:

• ng-click • ng-dblclick • ng-mousedown • ng-mouseup • ng-mouseenter • ng-mouseleave • ng-mousemove • ng-mouseover	• ng-keydown • ng-keypress • ng-keyup • ng-blur • ng-focus • ng-change

The ng-event directive from AngularJS will not override the element's original event (onclick, onfocus…), both will be executed.

Mouse Events

Mouse events occur when a mouse button is clicked on an element, in this order:

1. ng-mousedown
2. ng-mouseup
3. ng-click / ng-dblclick

Or when the cursor moves over an element, in this order:

1. ng-mouseover
2. ng-mouseenter
3. ng-mousemove
4. ng-mouseleave

You can add mouse events on any HTML element.

Syntax :- *<element* ng-click="*expression*"*></element>*

- ng-click :- directive tells AngularJS what to do when an HTML element is clicked.
- ng-dblclick :- directive tells AngularJS what to do when an HTML element is double-clicked.
- ng-mousedown :- directive tells AngularJS what to do when a mouse button is clicked on the specific HTML element
- ng-mouseup :- directive tells AngularJS what to do when a mouse click is finished

difference between 'ng-click' and 'ng-mousedown' directives

- ng-mousedown will trigger whenever any of your mouse buttons is clicked over the element (it doesn't wait till you release the key)
- ng-click will trigger when the left mouse button is pressed and released on that element.
  (press right click of mouse to see the difference)

```
<body ng-app="">
    <div ng-mousedown="count = count + 1" ng-click="count = count + 2"
      ng-init="count=0">Click me!</div>
    <h1>{{count}}</h1>
 </body>
```

Example :- (without Controller)

```
<button ng-click="count = count + 1" ng-init="count=0">
   Increment
</button>
<div>The Current Count is {{count}}</div>

<button ng-click="col ='blue'" ng-init="col='red'">
   Increment
</button>
<div style="color:{{col}}">The Current color is {{col}}</div>
```

- ng-mouseover :- directive tells AngularJS what to do when a mouse cursor moves over the specific HTML element.
- ng-mouseenter :- directive tells AngularJS what to do when a mouse cursor enters the specific HTML element.
- ng-mousemove :- directive tells AngularJS what to do when a mouse cursor moves over the specific HTML element.
- ng-mouseleave :- directive tells AngularJS what to do when a mouse cursor leaves the specific HTML element.

```
<h1 ng-mouseenter="col1='red'" ng-mouseleave="col1='green'" ng-init="count=10">
        Increment
 </h1>
 <div style="color:{{col1}}">color is {{col1}}</div>
```

Example :- (with Controller)

```
<div ng-app="myApp" ng-controller="myCtrl">
    <button ng-click="count = count + 1">Click me!</button>
    <p>{{ count }}</p>
</div>
    <script>
        var app = angular.module('myApp', []);
        app.controller('myCtrl', function($scope) {
        $scope.count = 0;
        });
    </script>
```

Example :- (with Controller using function)

```
<div ng-app="myApp" ng-controller="myCtrl">
        <button ng-click="myFunction()">Click me!</button>
        <p>{{ count }}</p>
</div>
    <script>
            var app = angular.module('myApp', []);
            app.controller('myCtrl', function ($scope) {
                $scope.count=0;
                $scope.myFunction = function() {
                $scope.count++;
                };
        });
    </script>
```

Example:- (function with parameter)

```
<div ng-app="myApp" ng-controller="myCtrl">
<button ng-click="myFunction('Hello World')">Click me!</button>
  <p>{{ msg }}</p>
</div>
    <script>
            var app = angular.module('myApp', []);
            app.controller('myCtrl', function ($scope) {

                $scope.myFunction = function(eveparam) {
                $scope.msg = eveparam;
                };
        });
    </script>
```

Event Listeners with Parameters

```
<div ng-app="myApp" ng-controller="myCtrl">
    <h2>AngularJs Events with Parameters Example</h2>
      <ul>
        <li ng-repeat="user in arrlist" ng-click="getuserdetails(user)">
        {{ user.name }}
        </li>
      </ul>
      <span style="color:Red">{{result1}}</span>
 </div>
<script>
            var app = angular.module('myApp', []);
           app.controller('myCtrl', function ($scope) {

             $scope.arrlist = [{"userid": 1,"name": "Suresh"},
                               {"userid": 2,"name": "Rohini"},
                               {"userid": 3,"name": "Praveen"}
                              ];

            $scope.getuserdetails = function (item) {
                  $scope.result1 = "Your Clicked on "+item.name;
             }
        });
  </script>
```

Keyboard Events

keyboard events occur when a user presses a keyboard key and one event when a user releases a keyboard key:

1. ng-keydown - executes when key is starting to press (this executes first and then keypress event executes)
2. ng-keypress - executes when key is pressed
3. ng-keyup - executes when key is left after pressing

Supported by <input>, <select>, <textarea>, and other editable elements.

Syntax :- <*element* ng-keypress/ng-keydown="*expression*"></*element*>

Value	Description
expression	An expression to execute when a key is pressed.

Syntax :- <*element* ng-keyup="*expression*"></*element*>

Value	Description
expression	An expression to execute when a keypress is finished.

KeyDown	This event is raised when a user presses a key.
KeyPress	This event is raised when the key or keys pressed result in a character. For example, a user presses SHIFT and the lowercase "a" keys, which result in a capital letter "A" character.
KeyUp	This event is raised when a user releases a key.

```
<input ng-keydown="count = count + 1" ng-init="count=0" />
<h1>{{count}}</h1>

<input ng-keypress="count = count + 1" ng-init="count=0" />
<h1>{{count}}</h1>
```

Example :- (with Controller using function)

```
<h2>Key Events</h2>
<div ng-app="app" ng-controller="KeyController">
   <p>Try writing something in the textbox below</p>
   <input type="text" ng-keydown="KeyDown()" ng-keypress="KeyPress()"
                  ng-keyup="KeyUp()" style="background:{{c}}"/>
   <p >Key down - {{ keydown }}</p>  <p>Key press - {{ keypress }}</p>
   <p >Key up - {{ keyup }}</p>
</div>
<script>
      var app = angular.module("app", []);
      app.controller("KeyController", function ($scope) {
         $scope.KeyDown = function () {
            $scope.keydown = " Key down executes ";
            $scope.c="red";
         }

         $scope.KeyPress = function () {
            $scope.keypress = " Key press executes ";
         }

         $scope.KeyUp = function () {
            $scope.keyup = " Key up executes ";
            $scope.c="green";
         }
      });
   </script>
```

ng-focus & ng-blur Directive

- ng-focus :- directive tells AngularJS what to do when an HTML element gets focus.
- ng-blur :- directive tells AngularJS what to do when an HTML element loses focus.

Supported by <a>, <input>, <select>, <textarea>, and the window object.

```
<body ng-app="">
   <input ng-blur="msg='Lost Focus (Blur) ';color1='green'"
        ng-focus="msg='Get Focus';color1 = 'red'" ng-init="msg='hello'" />
```

AngularJS Filters

Filters can be added in AngularJS to format data.

AngularJS provides filters to transform data:

- lowercase Format a string to lower case.
- uppercase Format a string to upper case.
- orderBy Orders an array by an expression.
- number Format a number to a string.
- currency Format a number to a currency format.
- date Format a date to a specified format.
- filter Select a subset of items from an array.
- LimitTo returns a new array or string containing only a specified number of elements

Filters can be added to expressions by using the pipe character |, followed by a filter.
{{expression | filter name}}

Adding Filters to Expressions

Uppercase / Lowercase Filter

The uppercase filter converts the string to upper case and lowercase filter converts the string to lower case.

Example

```
<div ng-app="myApp" ng-controller="personCtrl">
   <p>The name is {{ firstName | uppercase }}  {{ lastName | lowercase }}</p>
</div>
<script>
    angular.module('myApp', []).controller('personCtrl', function($scope) {
            $scope.firstName = "John",
            $scope.lastName = "Doe"
        });
```

```
</script>
```

OrderBy Filter

Filters are added to directives, like ng-repeat, by using the pipe character |, followed by a filter:

The orderBy filter sorts an array based on specified expression predicate.

For Ascending Order:- {{ expression | orderBy : predicate_expression}}

For Descending Order:- {{ expression | orderBy : - predicate_expression}}

Example

```
<div ng-app="myApp" ng-controller="namesCtrl">
        <p>Looping with objects:</p>
                <ul>
                 <li ng-repeat="x in names | orderBy:'country'">
                         {{ x.name + ', ' + x.country }}
                 </li>
                 </ul>
</div>
<script>
   angular.module('myApp', []).controller('namesCtrl', function($scope) {
        $scope.names = [ {name:'Jani',country:'Norway'},
                         {name:'Carl',country:'Sweden'},
                         {name:'Margareth',country:'England'},
                         {name:'Hege',country:'Norway'},
                         {name:'Joe',country:'Denmark'},
                         {name:'Gustav',country:'Sweden'}];
        });
</script>
```

Example

```
<body ng-app="myApp">
  <div ng-controller="myController">
    <select ng-model="SortOrder">
      <option value="+name">Name (asc)</option>
      <option value="-name">Name (dec)</option>
      <option value="+phone">Phone (asc)</option>
      <option value="-phone">Phone (dec)</option>
    </select>
    <ul ng-repeat="person in persons | orderBy:SortOrder">
      <li>{{person.name}} - {{person.phone}}</li>
    </ul>
  </div>
  <script>
    var myApp = angular.module('myApp', []);

    myApp.controller ("myController", function ($scope) {

      $scope.persons = [{ name: 'John', phone: '512-455-1276' },
                { name: 'Mary', phone: '899-333-3345' },
                { name: 'Mike', phone: '511-444-4321' },
                { name: 'Bill', phone: '145-788-5678' },
                { name: 'Ram', phone: '433-444-8765' },
                { name: 'Steve', phone: '218-345-5678' }]

    });
  </script>
```

```
</body>
```

Number Filter

A number filter formats numeric data as text with comma and specified fraction size.

`{{ number_expression | number:fractionSize}}`

If a specified expression does not return a valid number then number filter displays an empty string.

Example

```
Enter Amount: <input type="number" ng-model="amount" /> <br />
amount | number = {{amount | number}} <br />
amount | number:2 = {{amount | number:2}} <br />
amount | number:4 = {{amount | number:4}} <br />
amount | number = <span ng-bind="amount | number"></span>
```

Enter Amount: 1234.567893
amount | number = 1,234.568
amount | number:2 = 1,234.57
amount | number:4 = 1,234.5679
amount | number = 1,234.568

Currency Filter

The currency filter formats a number value as a currency. When no currency symbol is provided, default symbol ($) for current locale is used.

`{{ expression | currency : 'currency_symbol' : 'fraction'}}`

Rs symbol code : ₹

Example

```
<div ng-controller="myController">
    Default currency: {{person.salary | currency}} <br />
    Custom currency identifier: {{person.salary | currency:'Rs.'}} <br />
    No Fraction: {{person.salary | currency:'Rs.':0}} <br />
    Fraction 2: <span ng-bind="person.salary| currency:'GBP':2"></span>
</div>
<script>
    var myApp = angular.module('myApp', []);
    myApp.controller("myController", function ($scope) {
        $scope.person = { firstName: 'James', salary: 100000}
    });
</script>
```

Default currency: $100,000.00
Custom currency identifier: Rs.100,000.00
No Fraction: Rs.100,000
Fraction 2: GBP100,000.00

Date Filter

In angularjs, date filter is used to convert date to string based on the given format. In angularjs, we can convert date to multiple formats using date filter.

```
{{ date_expression | date : 'format'}}
```

date format either as Date object or date string formats like yyyy-MM-dd.

```
<div ng-app="" ng-init="today1='2019-04-22'">
      date: {{today1 | date}} <br />
      Short date: {{today1| date:'shortDate'}} <br />
      Long date: {{today1 | date:'longDate'}} <br />
      Year: {{today1 | date:'yyyy'}} <br />
</div>
<script>
   var app = angular.module('myApp', []);
   app.controller('datCtrl', function($scope) {
        $scope.today = new Date();
    });
</script>
```

Format	Expression	Result
Date	{{sampledate | date }}	Feb 28, 2016
Short	{{sampledate | date:"short" }}	2/28/16 7:33 PM
Medium	{{sampledate | date:"medium" }}	Feb 28, 2016 7:32:55 PM
shortDate	{{sampledate | date:"shortDate" }}	2/28/16
longDate	{{sampledate | date:"longDate" }}	February 28, 2016
fullDate	{{sampledate | date:"fullDate" }}	Sunday, February 28, 2016
mediumTime	{{sampledate | date:"mediumTime" }}	7:37:34 PM
shortTime	{{sampledate | date:"shortTime" }}	7:37 PM
yyyy (Year)	{{sampledate | date:"yyyy" }}	2016
yy (Year)	{{sampledate | date:"yy" }}	16
y (Year)	{{sampledate | date:"y" }}	2016
MMMM (Month)	{{sampledate | date:"MMMM" }}	February
MMM (Month)	{{sampledate | date:"MMM" }}	Feb
MM (Month)	{{sampledate | date:"MM" }}	02
M (Month)	{{sampledate | date:"M" }}	2
dd (Date)	{{sampledate | date:"dd" }}	28

Format	Expression	Result
d (Date)	{{sampledate | date:"d" }}	28
EEEE (Day)	{{sampledate | date:"EEEE" }}	Sunday
EEE (Day)	{{sampledate | date:"EEE" }}	Sun
HH (24 Hours Format)	{{sampledate | date:"HH" }}	19
H (24 Hours Format)	{{sampledate | date:"H" }}	19
hh (12 Hours Format)	{{sampledate | date:"hh" }}	07
h (12 Hours Format)	{{sampledate | date:"h" }}	7
mm (Minute)	{{sampledate | date:"mm" }}	16
m (Minute)	{{sampledate | date:"m" }}	16
sss (Milli Seconds)	{{sampledate | date:"sss" }}	501
ss (Seconds)	{{sampledate | date:"ss" }}	45
s (Seconds)	{{sampledate | date:"s" }}	45
a (AM/PM)	{{sampledate | date:"a" }}	PM
Z (TimeZone)	{{sampledate | date:"Z" }}	0530
ww (Week of year)	{{sampledate | date:"ww" }}	09
w (Week of year)	{{sampledate | date:"w" }}	9

Filter

In angulajs, filter is used get filtered subset of items from array items list based on user input filter key text.

{{ *arrayexpression* | filter : *expression* : *comparator* }}

{{name |filter:'{name:'a'}'}}

comparator as true, it will ensure that only items which are an exact match to the expression are returned.

```
<script>
   var app = angular.module('myApp', []);
   app.controller('datCtrl', function($scope) {
       $scope.names= ['Aryan','Ajeet','Sonoo','Joe','Lily','Lucy','Mary',
                      'Kali'];
       });
</script>
<div ng-app="myApp" ng-controller="datCtrl">
```

```
    Enter Name to Filter:<input ng-model="searchtxt" type="text">
     <ul>
        <li ng-repeat="x in names | filter : searchtxt:true">
            {{ x }}
        </li>
     </ul>
     <ol>
         <li ng-repeat="x in names | filter : searchtxt">
                {{ x }}
         </li>
     </ol>
</div>
```

limitTo Filter

limitTo filter returns a new array or string containing only a specified number of elements. It chooses the elements either the beginning or the end of the source array string or number, as specified by the value and sign (positive or negative) of limit.

limitTo filter used for array: It returns an array, containing only a specified number of elements.
limitTo filter used for string: It returns a string, containing only specified number of characters.
limitTo filter used for numbers: It returns a string, containing only specified number of digits.

```
{{ limitTo_expression | limitTo : limit : begin}}
```

input: It specifies array/array-like, string or number to be limited.

limit: It specifies the length of the returned array or string.

- If the limit number is positive, it chooses the items from the beginning of the source array/string.
- If the number is negative, it chooses the items from the end of the source array/string.

begin: It is optional. It specifies the index at which to begin limitation. By default, its value is 0.

```
<div ng-app="myApp" ng-controller="sizeCtrl">
        <p>It displays only the first 3 cars because limit is 3 (positive).</p>
        <ul>
            <li ng-repeat="x in cars | limitTo : 3">{{x}}</li>
        </ul>

        <p>It displays only the last 3 cars because limit is -3 (negative).</p>
            <ul>
                <li ng-repeat="x in cars | limitTo : -3">{{x}}</li>
            </ul>
</div>
```

```
<script>
        var app = angular.module('myApp', []);
        app.controller('sizeCtrl', function($scope) {
                $scope.cars = ["Audi", "BMW", "Tata", "Mahindra", "Ford", "Volvo"];
        });
</script>
```

AngularJS form Validations:

AngularJS performs **form validation** on the client side. AngularJS monitors the state of the form and input fields (input, text-area, select), and notify the user about the current state. AngularJS also holds information about whether the input fields have been touched, modified, or not. Form input fields have the following states:

- **$untouched:** It shows that field has not been touched yet.
- **$touched:** It shows that field has been touched.
- **$pristine:** It represents that the field has not been modified yet.
- **$dirty:** It illustrates that the field has been modified.
- **$invalid:** It specifies that the field content is not valid.
- **$valid:** It specifies that the field content is valid.

Example:

```
<table border = "0">
    <tr>
      <td>Enter first name:</td>
      <td><input name = "firstname" type = "text" ng-model = "firstName" required>
        <span style = "color: red" ng-show = "studentForm.firstname.$dirty && studentForm.firstname.$invalid">
          <span ng-show = "studentForm.firstname.$error.required">First Name is required.</span>
        </span>
      </td>
    </tr>

    <tr>
      <td>Enter last name: </td>
      <td><input name = "lastname"  type = "text" ng-model = "lastName" required>
        <span style = "color: red" ng-show = "studentForm.lastname.$dirty && studentForm.lastname.$invalid">
          <span ng-show = "studentForm.lastname.$error.required">Last Name is required.</span>
        </span>
      </td>
    </tr>

    <tr>
      <td>Email: </td><td><input name = "email" type = "email" ng-model = "email" length = "100" required>
        <span style = "color: red" ng-show = "studentForm.email.$dirty && studentForm.email.$invalid">
          <span ng-show = "studentForm.email.$error.required">Email is required.</span>
          <span ng-show = "studentForm.email.$error.email">Invalid email address.</span>
        </span>
      </td>
    </tr>
     <tr>
      <td>
        <button ng-click = "reset()">Reset</button>
      </td>
      <td>
        <button ng-disabled = "studentForm.firstname.$dirty &&
          studentForm.firstname.$invalid || studentForm.lastname.$dirty &&
          studentForm.lastname.$invalid || studentForm.email.$dirty &&
          studentForm.email.$invalid" ng-click="submit()">Submit</button>
      </td>
    </tr>
</table>
```